

Unity (C#) - Build & Publish Guide

Turning the Editor project into a standalone Kestrel Watch executable

VIDEO GAME CREATION

This guide walks through producing a double-click-able Windows build of Kestrel Watch straight out of the Unity Editor - no code changes required. Do this after the project passes every in-Editor Play Test section; a bug that shows up in Play mode will still be there in the standalone build, so there's nothing to gain by building early and the build step takes several minutes each time.

0. Before You Build

- Run through the Manual Play Test Plan's Sections 0-10 in the Editor first (Play mode). Don't build until those pass - a standalone build is slower to test than Play mode (every change means a full rebuild, not just pressing Play again), so catch what you can catch early.
- Open the Console window (Window > General > Console) and make sure it's clear of errors (red) before building. Warnings (yellow) won't block a build but should already be at zero per the earlier compiler-warning cleanup.
- Save the scene (Ctrl/Cmd+S) and the project (File > Save Project) immediately before building, so the build matches exactly what you just tested.

1. Player Settings

Player Settings controls everything about how the finished .exe identifies and presents itself - window title, taskbar icon, folder names it creates. Open it from Edit > Project Settings > Player (or File > Build Settings > Player Settings... from the Build Settings window covered in Section 2).

Field	Set it to	Why
Company Name	`Doers of Stuff`	Used to name the folder Unity creates for save data (not used by this game, but still set it - an empty Company Name produces an awkward default folder name).
Product Name	`Kestrel Watch`	This is the window title bar text and the .exe's file name - matches the in-game title screen.
Default Icon	`Optional - default Unity icon is fine`	A custom icon needs a square .png (1024x1024 is safe) dragged into this field under Icon > Default Icon. Not required for Code Complete.

Field	Set it to	Why
Resolution and Presentation > Fullscreen Mode	`Windowed`	Easier to test and to Alt+Tab out of during development. Fullscreen-related options (Borderless, Exclusive) are fine to explore later but aren't required.
Resolution and Presentation > Default Screen Width/Height	`1280 x 720 (or whatever you've been testing in the Game view)`	Sets the window size the game opens at.
Resolution and Presentation > Resizable Window	`Checked`	Lets a player resize the window; the game doesn't depend on a fixed resolution since the Player's movement is clamped by camera bounds, not hardcoded pixel values.

Company Name and Product Name have not been set yet on a fresh project (Unity's defaults are "DefaultCompany" and the project's folder name) - this is the first time this project needs them, so set both now.

2. Build Settings - Adding the Scene

1. Open File > Build Settings (or File > Build Profiles in newer Unity versions - either one gets you to the same scene list and Build button).
2. Look at the "Scenes In Build" list. If it's empty, the build will produce a window that opens to a black screen and nothing else - this is the single most common reason a first build "doesn't work."
3. With your game's scene open in the Editor, click "Add Open Scenes." It should appear in the list, checked, at index 0 (the top one is what loads first - there's only one scene in this project, so that's automatic).
4. Confirm Platform is set to "Windows, Mac, Linux" with Target Platform "Windows" (or your own OS, if you're not on Windows) and Architecture "x86_64." If a different platform is selected, click it in the list on the left and press "Switch Platform" - this can take a minute the first time.

3. Building the Executable

5. Click "Build" (not "Build And Run" the first time - it's fine either way, but doing a plain Build first means you can look at the output folder before anything launches).
6. Unity asks where to save it. Create a new folder for this - something like Builds\KestrelWatch_Win64 inside (or next to) the project, NOT inside the Assets folder. Give the build itself a file name when prompted, e.g. KestrelWatch.exe.
7. Unity compiles and packages the build - this takes anywhere from under a minute to several minutes depending on your machine. A progress bar shows in the bottom-right of the Editor.
8. When it finishes, Unity opens the output folder automatically (or you can browse to the folder you chose). You should see the .exe plus a same-named _Data folder next to it, and a couple of supporting files (UnityPlayer.dll, UnityCrashHandler64.exe) - all of this together IS the build. None of it is optional; the .exe will not run without its _Data folder sitting right next to it.

4. Running and Testing the Build

- Double-click the .exe. A small Unity resolution/graphics dialog may appear first (depends on the Resolution and Presentation settings above) - Play through it, or it opens straight into the game if that dialog was disabled.
- Run through the Manual Play Test Plan's Sections 1-11 again here, in the standalone build this time. This is the "standalone executable" pass your test plan calls for - everything that passed in the Editor should behave identically, since this is the same compiled game, just packaged on its own.
- There is no Stop button outside the Editor. To close the game, use the window's own close button (X) or Alt+F4. Nothing in this project currently binds a quit key or calls Application.Quit() - that's expected, not a bug, since the Editor's Stop button made one unnecessary during development. Worth knowing before you're in the Test Plan looking for a way out of a fullscreen window.
- Standalone builds write their own log file instead of printing to a visible Console window. If something misbehaves only outside the Editor, the log is at %USERPROFILE% \AppData\LocalLow\Doers of Stuff\Kestrel Watch\Player.log on Windows (path uses the Company Name / Product Name from Section 1) - open it in any text editor.

5. Known Gotchas

- Black screen on launch: almost always an empty "Scenes In Build" list (Section 2) - the Editor still works fine in this case because Play mode doesn't need the build list, only an actual build does.
- Windows SmartScreen warning ("Windows protected your PC"): expected for any unsigned .exe from an indie/student project - click "More info" then "Run anyway." This is not a sign of a broken or unsafe build, just that it isn't digitally signed (code-signing certificates cost money and aren't part of this course).
- Antivirus false positives: occasionally flags newly-built, unsigned .exe files. Same cause as SmartScreen above - safe to allow for a build you just made yourself.
- Sharing the build with someone else: zip the WHOLE output folder (the .exe and its _Data folder together) before sending it anywhere - sending just the .exe on its own will not run.
- Re-building after a code change: no extra steps beyond repeating Section 3 - Unity recompiles automatically. You don't need to redo Player Settings or Build Settings unless you're changing one of those specific settings.

6. Declaring Done

Once the Manual Play Test Plan passes cleanly both in the Editor (Play mode) and in this standalone build, Unity is fully Code Complete and ready to serve as the master model for the next engine. Keep the build folder (or at least the .exe) around as the reference "known-good" build to compare future engines' output against.